Diagrama Dinâmico de Fluxo Energético (DDFE)

Tutorial do Desenvolvedor: Engine de Layout e Visualização (Power Query + Deneb)

Este documento técnico detalha a arquitetura, a engenharia de dados e os métodos de renderização visual empregados no projeto de visualização de fluxo . O objetivo é fornecer uma base sólida para futuros desenvolvedores que desejem manter, otimizar ou expandir as capacidades do sistema.

1. Visão Geral da Arquitetura

A solução abandona visuais de Sankey tradicionais em favor de uma engine customizada dividida em duas camadas principais :

- **Back-end Gráfico (Power Query / Linguagem M):** Responsável por todo o cálculo geométrico. Ele lê as topologias, aloca coordenadas X e Y absolutas, resolve colisões e identifica loops de retroalimentação (Backflows) .
- **Front-end de Renderização (Deneb / Vega):** Recebe um "dataset" achatado contendo as coordenadas pré-calculadas e o renderiza usando primitivas vetoriais (retângulos, linhas, símbolos). Ele também gerencia a interatividade (hitboxes) .

2. Dissecando a Engenharia de Dados (Power Query)

O script M é extenso e altamente estruturado. Abaixo estão as etapas lógicas críticas que merecem atenção em manutenções futuras.

2.1. Lógica de Hub e Linkagem (Input_Tech)

Para lidar com conversões diretas de combustível sem uma tecnologia explícita mapeada, o sistema usa uma "Lógica de Hub". Se uma linha possui entrada e saída mas o campo tech está vazio, o sistema gera dinamicamente um identificador como LINK_HUB_[fuel_in] ou LINK_ORFAO_[Guid] . Esses nós são marcados com a flag IsHidden = true , instruindo o Vega a não desenhar a caixa da tecnologia, apenas a

linha de conexão.

2.2. Cálculo do Eixo X e Paredes Geométricas (TechLayout)

O eixo horizontal não é apenas baseado em estágios, mas em "Paredes" (Limites Max e Min de cada estágio). A Query identifica Max_Stage_In e Min_Stage_Out para cada tecnologia . O posicionamento x_tech é calculado verificando três casos :

- **Caso 1 (Tem Input e Output):** Se for fluxo normal (Forward), centraliza entre a parede direita do Input e a esquerda do próximo Input . Se for Backflow (Input >= Output), a lógica reverte as referências espaciais .
- **Caso 2 (Apenas Input):** Posiciona à direita do combustível de origem .
- **Caso 3 (Apenas Output):** Posiciona à esquerda do combustível de destino .

2.3. O Eixo Y e os "Ghost Nodes" (Nós Fantasmas)

Este é o núcleo do algoritmo de roteamento sem sobreposição . Para conexões longas (Long Haul) que cruzam múltiplos estágios, a linha desenhada não pode atravessar outras caixas. O projeto resolve isso gerando **Ghost Nodes** .

```
// Trecho conceitual da lógica de fantasmas
GeraGhostsGeometricos = Table.AddColumn(FiltraLongHaul, "GhostGrid", each
   let
      Start = List.Min({[x_tech], [x_fuel_out]}),
      End = List.Max({[x_tech], [x_fuel_out]}),
      GridsNoCaminho = List.Select(ListaGridsOcupados, (g) => g > (Start + 25) and
g < (End - 25))
   in
      GridsNoCaminho
)
```

Esses nós invisíveis "inflam" a densidade local (QtdItens) daquela raia (swimlane), garantindo que o cálculo de Y_Base_Raia reserve espaço suficiente para que a linha horizontal transite livremente .

2.4. Motor de Rastreabilidade (TraceabilityEngine)

Embora atualmente comentado para ganhos de performance no carregamento final , o projeto possui um mecanismo nativo de busca em Grafos (Upstream e Downstream) usando recursão M (List.Generate e Record.Field) . Se ativado, ele cria Hash Maps de arestas e gera colunas Ancestors_Path e Descendants_Path ,

permitindo que o usuário clique em um nó e ilumine toda a cadeia produtiva.

3. Front-end Visual (Deneb / Vega)

O JSON do Vega é impulsionado por um sistema de signals (sinais reativos) que respondem a eventos do mouse para fornecer destaque interativo (Highlighting) .

Sinal (Signal)	Gatilho (Events)	Efeito na Opacidade
sel_fuel_color	@fuel_hitbox:click	Destaca barras verticais e linhas de conexão com cor idêntica .
sel_tech_color	@tech_box:click	Destaca tecnologias (caixas) .
sel_line_color	@input_hitbox:click	Ilumina especificamente os fluxos horizontais de entrada .

3.1. Hitboxes vs. Visuais

Você notará a existência de conjuntos duplos de dados e marcações (ex: input_normal_ds alimentando input_hitbox e input_visual) . Isso é uma técnica clássica de UI: a hitbox possui strokeWidth: 45 e strokeOpacity: 0.001 , criando uma área de clique grossa e invisível ao redor de uma linha fina de 2px (o visual real). Sempre mantenha esse padrão ao adicionar novos elementos interativos.

3.2. Tratamento de Backflow

O Vega usa quatro conjuntos de regras separadas para desenhar o roteamento de fluxos retrógrados (bf_travel, bf_down, bf_enter, bf_point) . Eles utilizam coordenadas auxiliares calculadas no dataset, como yBridge (para passar por cima das caixas) e xTurn (para a curva) . Qualquer mudança na altura das caixas (techH)

exigirá ajuste proporcional no sinal hookGap .

4. Próximos Passos Sugeridos

- **Parametrização Deneb:** Externalizar as variáveis de padding e tamanho de caixa do Vega para campos do Power BI.
- **Otimização do Graph:** Avaliar a reativação da TraceabilityEngine utilizando funções C# personalizadas ou M avançado para evitar timeout em datasets massivos.